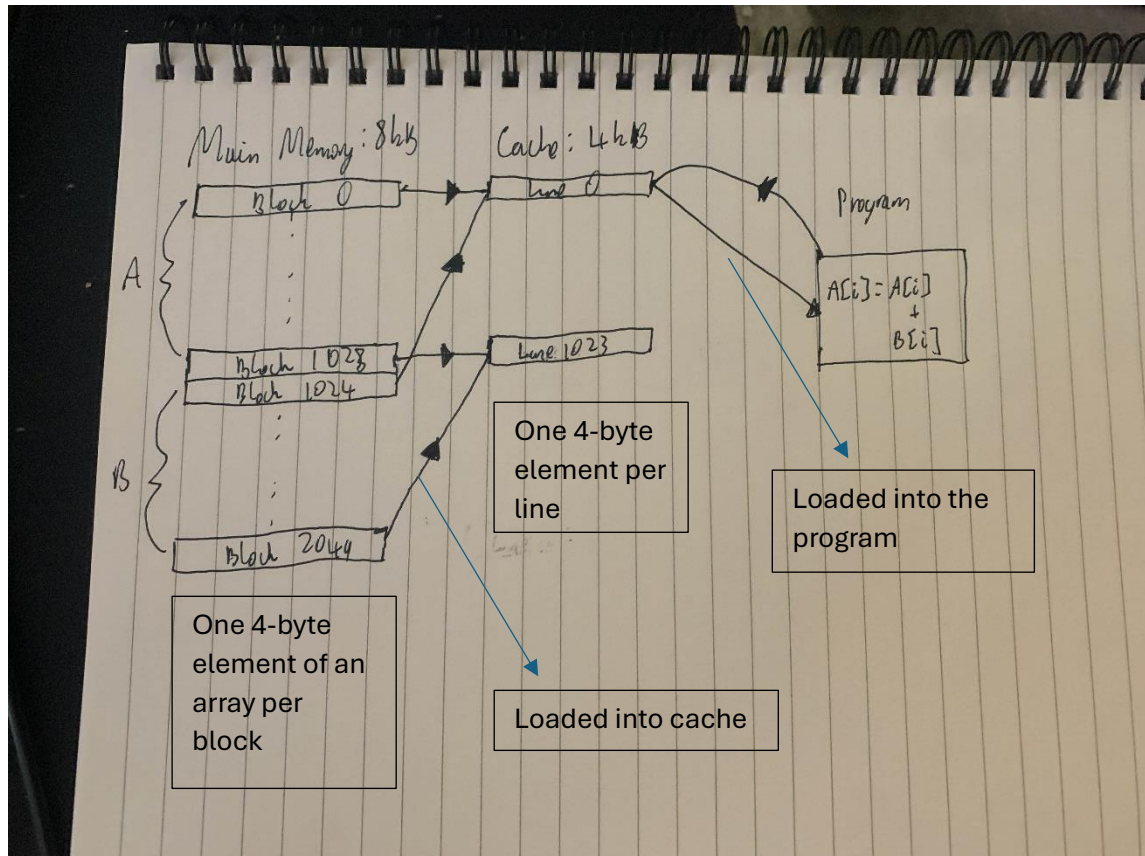




Algorithm:

```
for (int i = 0; i < 1024; i++)
{
    A[i] = A[i] + B[i];
}
```

With a direct mapped cache policy, the cache is divided into lines where data can be stored. The mapping works based on the original memory address, and it directly maps data stored in main memory into a line of memory in the cache, and which line it maps to depends on factors such as the size of the memory and the size of the cache.

In our case, the main memory is 8kb in size, and the cache is 4kb in size. For a unique memory address for each 4-byte block of memory, we need a 13-bit address. If we assume a cache line size of 4 bytes (i.e. one integer), then there are 1024 cache lines. Each address is split into bits for different purposes but most importantly the index, which in our case will be 10 bits long, and the tag which is 1 bit long.

The algorithm will suffer poor performance with this cache policy due to elements of the different arrays being mapped to the same cache line. Each block of memory can only be loaded into one cache line. The first step in the algorithm is to access the element at $A[0]$, which the CPU will load into the cache line 0 after failing to find it in the cache (a cache miss). After doing so, it will attempt to access $B[0]$ which again will miss, so will attempt to load the data into the cache which maps to the same cache line as $A[0]$. This causes an issue as only one element can be stored in a cache line, so $A[0]$ is evicted and $B[0]$ is replaced. Then the algorithm

needs to store the resulting sum back into A[0], which means that the cache has to reload A[0] again and evict B[0]. Eventually A[0] will be flushed from the cache and back into main memory. This process continues for every element of the list.

This process has poor performance due to the frequency of misses and evictions that happens within the small cache size. As the tag is only 1 bit long, the cache cannot access unique memory addresses that are in the same cache line causing the process of missing and eviction described.

A hardware change for this will be increasing the cache size such that elements of both arrays can be stored in the cache uniquely, leading to less frequent misses. Another change is implementing a different cache policy such as fully associative, which has better utilisation of the cache and a more effective way of replacing data when a cache miss occurs.

A software fix will be changing the algorithm such that it assigns the summation into another array i.e:

```
for (int i = 0; i < 1024; i++)
{
    C[i] = A[i] + B[i];
}
```

which will reduce the number of conflicts that arise when reloading A[i] to write to.

Another will be tiling the loop such that the cache is used more efficiently i.e.:

```
for (int i = 0; i < 1024; i+=32)
{
    for (int j = i; j < i + 32; j++)
    {
        A[j] = A[j] + B[j];
    }
}
```

Part b

If a L1 miss then a L2 hit is equal to 10 clock cycles, and L1 and L2 miss and main memory “hit” is 100 clock cycles then:

1000 references

960 L1 hits = 960 clock cycles

20 L2 hits = 200 clock cycles

20 main hits = 2000 clock cycles

Average is $(2000 + 960 + 200) / 1000 = 3.16$ clock cycles

If a L1 miss then a L2 hit is equal to (1+10) clock cycles, and L1 and L2 miss and main memory “hit” is (1+10+100) clock cycles then:

1000 references

960 L1 hits = 960 clock cycles

20 L2 hits = $(20 \times 11) = 220$ clock cycles

20 main hits = $(20 \times 111) = 2220$ clock cycles

Average is $(960 + 220 + 2220) / 1000 = 3.4$ clock cycles